



Práctica 3.2. DAST — OWASP ZAP
OWASP ZAP contra aplicación ASP.NET Core 10

Vazquez Rangel Guillermo Larzen

2024171016

IDGS17

Gerardo Ramírez Villarreal

Seguridad en el Desarrollo de Aplicaciones

Objetivo

Ejecutar un análisis dinámico de seguridad (DAST) con OWASP ZAP contra el proyecto ASP.NET Core 10 (VulnerableApp) trabajado en prácticas anteriores, dentro de un entorno Docker completamente aislado. Se documentan los hallazgos identificados con su nivel de riesgo, CWE y CVSS estimado, se verifica el efecto de las correcciones aplicadas en la Práctica 2 (P2) sobre los resultados del escaneo, y se integra ZAP como job de DAST en el pipeline de GitHub Actions.

Fase 1 — Preparación del entorno

El escaneo se ejecutó en una red Docker interna, sin acceso a Internet, siguiendo el aislamiento exigido por la práctica. El alcance de la prueba fue exclusivamente el contenedor vulnerable_app.

Red Docker aislada

```
docker network create --internal pentest-net
docker network inspect pentest-net | grep Internal
# "Internal": true
```

Para estandarizar el escaneo se utilizó el Automation Framework de ZAP mediante el archivo zap.yaml, que define el contexto, el spider, la espera del passive scan y la generación del reporte HTML:

```
env: contexts: - excludePaths: [] name: baseline urls: - http://vulnerable_app:8080 parameters:
failOnError: true jobs: - type: passiveScan-config parameters: {enableTags: false,
maxAlertsPerRule: 10} - type: spider parameters: {url: http://vulnerable_app:8080,
maxDuration: 1} - type: passiveScan-wait - type: outputSummary parameters: {format: Long,
summaryFile: /home/zap/zap_out.json} - type: report parameters: reportDir: /zap/wrk/
reportFile: reporte_vulnerabilidades.html reportTitle: ZAP Scanning Report template:
traditional-html
```

Se construyeron y levantaron los contenedores vulnerable_app y zap sobre la red interna pentest-net, quedando ambos en estado Started dentro de la misma red aislada, tal como se observa en la evidencia 1.

Fase 2 Escaneo pasivo y activo con ZAP

2.1 Escaneo pasivo (baseline)

Se ejecutó zap-baseline.py contra `http://vulnerable_app:8080`, generando el reporte `reporte_vulnerabilidades.html`. Este escaneo recorre la aplicación (spider) y analiza pasivamente las respuestas HTTP, sin enviar payloads de ataque:

```
$ docker exec -it zap zap-baseline.py -t http://vulnerable_app:8080 -r
reporte_vulnerabilidades.html Using the Automation Framework Total of 3 URLs PASS:
Vulnerable JS Library (Powered by Retire.js) [10003] PASS: Cookie No HttpOnly Flag [10010]
PASS: Cookie Without Secure Flag [10011] PASS: Information Disclosure - Debug Error
Messages [10023] PASS: Off-site Redirect [10028] PASS: User Controllable HTML Element
Attribute (Potential XSS) [10031] ...
```

La mayoría de las reglas pasivas relacionadas con cookies, fugas de información, XSS reflejado controlado por el usuario y librerías JS vulnerables resultaron en PASS, es decir, no se identificaron hallazgos en esas categorías.

2.2 Escaneo activo completo (full scan)

Adicionalmente se ejecutó zap-full-scan.py, que además del análisis pasivo envía payloads activos (SQL Injection, XSS, etc.) contra la aplicación, generando `fullscan-report.html/.xml/.json`.

2.3 Resultado agregado de ambos escaneos

Tanto el escaneo pasivo (`reporte_vulnerabilidades.html`) como el escaneo activo completo (`fullscan-report.*`) arrojaron el mismo resumen de alertas:

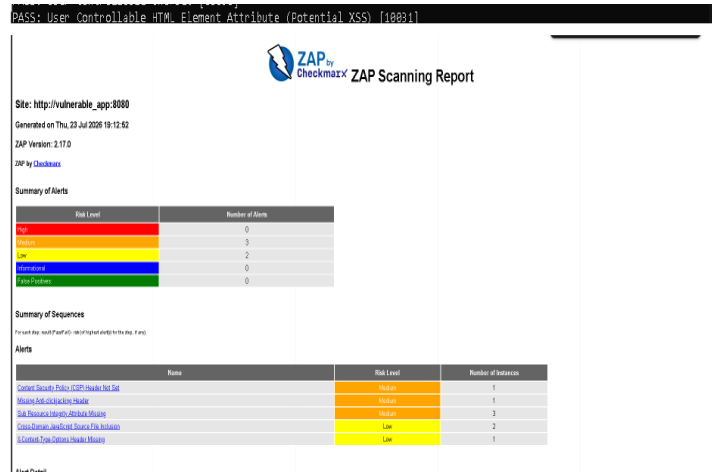
Nivel de riesgo	Nº de alertas	Color ZAP
High	0	—
Medium	3	Naranja
Low	2	Amarillo
Informational	0	—
False Positives	0	—

Sitio analizado: `http://vulnerable_app:8080` · ZAP Version: 2.17.0 · Generado: 23 Jul 2026, 19:34 UTC.

Evidencia 1 — Ejecución del baseline scan

```
huesos@desktop-HUGOOSV: ~/Documents/VulnerableApp (logs_3)
$ docker exec -it zap zap-baseline.py -t http://vulnerable_app:8080 -r reporte_vulnerabilidades.html
Using the Automation Framework
Total of 3 URLs
PASS: Vulnerable JS Library (Powered by Retire.js) [10003]
PASS: In Page Banner Information Leak [10000]
PASS: Cookie No HttpOnly Flag [10010]
PASS: Cookie Without Secure Flag [10011]
PASS: Re-examine Cache-control Directives [10015]
PASS: Content-Type Header Missing [10019]
PASS: Information Disclosure - Debug Error Messages [10023]
PASS: Information Disclosure - Sensitive Information in URL [10024]
PASS: Information Disclosure - Sensitive Information in HTTP Referrer Header [10025]
PASS: Information Disclosure - Suspicious Comments [10027]
PASS: Off-site Redirect [10020]
PASS: Cookie Poisoning [10020]
PASS: User Controllable Character [10030]
```

Evidencia 2 — Resumen del reporte ZAP



Fase 3 — Análisis y documentación de hallazgos

No se identificaron hallazgos de riesgo High en ninguno de los dos escaneos (pasivo y activo completo). Los 5 hallazgos obtenidos —todos de riesgo Medium o Low— corresponden a cabeceras de seguridad HTTP ausentes y a la falta de verificación de integridad de recursos de terceros (SRI). A continuación se documenta la tabla completa exigida por la práctica (todos los hallazgos Medium, ya que no hubo High):

#	Hallazgo	Riesgo ZAP	CWE	CVSS est.	URL afectada	Evidencia	Remediación
1	Content Security Policy (CSP) Header Not Set	Medium (High)	CWE-693	5.3 (Medium)	http://vulnerable_app:8080/ (2 instancias)	Ausencia del header Content-Security-Policy en la respuesta a GET /	Configurar el header Content-Security-Policy con directivas restrictivas (default-src 'self'; script-src; style-src; frame-ancestors 'self').
2	Missing Anti-clickjacking Header	Medium (Medium)	CWE-1021	4.3 (Medium)	http://vulnerable_app:8080/ (2 instancias)	No se envía X-Frame-Options ni CSP con frame-ancestors en la respuesta.	Agregar X-Frame-Options: DENY (o SAMEORIGIN si se requiere) o la directiva CSP frame-ancestors 'self'.

#	Hallazgo	Riesgo ZAP	CWE	CVSS est.	URL afectada	Evidencia	Remediación
3	Sub Resource Integrity Attribute Missing	Medium (High)	CWE-345	5.9 (Medium)	http://vulnerable_app:8080/ (5 instancias — Bootstrap CSS/JS, jQuery)	<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" /> sin atributo integrity; <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script> sin integrity.	Añadir atributos integrity (hash SRI, ej. sha384-...) y crossorigin="anonymous" a cada <link>/<script> cargado desde un CDN externo.
4	Cross-Domain JavaScript Source File Inclusion	Low (Medium)	CWE-829	3.7 (Low)	http://vulnerable_app:8080/ (4 instancias)	Scripts cargados desde dominios de terceros: cdn.jsdelivr.net y code.jquery.com.	Alojar localmente las librerías críticas o restringir mediante CSP (script-src) los orígenes permitidos, combinado con SRI.
5	X-Content-Type-Options Header Missing	Low (Medium)	CWE-693	3.1 (Low)	http://vulnerable_app:8080/ (2 instancias)	El header X-Content-Type-Options no está presente / no está fijado a nosniff.	Configurar X-Content-Type-Options: nosniff en todas las respuestas del servidor/aplicación.

Análisis detallado del hallazgo de mayor riesgo

El hallazgo con el CVSS estimado más alto es Sub Resource Integrity Attribute Missing (CVSS 5.9, Medium), por lo que se documenta a continuación en detalle.

1. Nombre y clasificación OWASP

Sub Resource Integrity Attribute Missing — corresponde a A08:2021 – Software and Data Integrity Failures del OWASP Top 10, dado que la aplicación confía en contenido ejecutable (JS/CSS) servido por terceros (CDNs) sin verificar su integridad.

2. Request HTTP

El hallazgo es detectado de forma pasiva por ZAP: no se envía un payload de ataque, sino que se analiza el HTML devuelto por una solicitud normal de navegación. Los reportes JSON/XML exportados por ZAP no incluyen la traza cruda de la petición/respuesta (solo quedaría disponible en el HTTP History de la GUI de ZAP); la solicitud registrada en el hallazgo es:

GET / HTTP/1.1 Host: vulnerable_app:8080

3. Response HTTP que reveló la vulnerabilidad

Dentro del cuerpo HTML de la respuesta 200 OK, ZAP identificó las siguientes etiquetas <link>/<script> apuntando a un CDN externo sin atributo integrity:

```
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" /> <script
src="https://code.jquery.com/jquery-3.7.1.min.js"></script> <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
```

4. Por qué esta respuesta confirma la vulnerabilidad

El atributo integrity permite al navegador calcular un hash (SHA-384, por ejemplo) del recurso descargado y compararlo contra el valor declarado en el HTML; si no coinciden, el navegador rechaza ejecutar/aplicar el recurso. Como ninguna de las etiquetas <link> o <script> externas incluye integrity, el navegador ejecuta ciegamente cualquier contenido que el CDN entregue en ese momento, sin forma de detectar una modificación.

5. Qué podría obtener un atacante real

Si un atacante comprometiera el CDN (cdn.jsdelivr.net o code.jquery.com) o interceptara la conexión hacia él, podría sustituir el JavaScript servido por código malicioso que se ejecutaría en el navegador de cada usuario de la aplicación con los mismos privilegios que el resto del contenido de la página (robo de cookies de sesión, keylogging de formularios, redirección a sitios de phishing, minería en el navegador, etc.), de forma equivalente a un XSS persistente pero originado en la cadena de suministro y no en el propio código de la aplicación.

6. Código que elimina la vulnerabilidad

Agregar los atributos integrity y crossorigin="anonymous" con el hash SRI publicado por el proveedor del CDN:

```
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+
ALEwIH" crossorigin="anonymous"> <script src="https://code.jquery.com/jquery-3.7.1.min.js"
integrity="sha256-/JqT3SQfawRcv/BIHPTThkBvs0OEvtFFmqPGmkzjNHOo="
crossorigin="anonymous"></script>
```

Los valores de integrity deben tomarse de la fuente oficial del recurso (jsDelivr / jQuery CDN) en el momento de fijar la versión, y no generarse manualmente.

Verificación de las correcciones de la Práctica 2 (P2)

El escaneo activo completo (zap-full-scan.py) incluye reglas activas de SQL Injection y Cross-Site Scripting que envían payloads reales contra los formularios y parámetros de la aplicación (Search/Index, Comment/AddComment, /api/user/{id}, /Auth/Login, entre otros). Este mismo escaneo es el que se documenta en este reporte como el estado "después" de aplicar las correcciones de P2 (SQL parametrizado, Html.Encode/codificación de salida, validación de entrada).

El resultado confirma que las correcciones son efectivas: fullscan-report.json/.xml/.html no registran ninguna alerta de tipo SQL Injection, Cross Site Scripting (Reflected/Persistent) ni Path Traversal (0 hallazgos High), pese a que el motor de ataque de ZAP ejecutó dichas reglas activas sobre la aplicación. Esto es consistente con lo observado en la Práctica 3 — Logs, Parte 2, donde los 20 intentos de SQL Injection (1' OR '1'='1) y los 30 intentos de XSS (<script>alert(1)</script>) quedaron registrados en el log de la aplicación como intentos detectados/neutralizados, sin que ninguno derivara en una respuesta vulnerable observable por ZAP.

Los únicos hallazgos que persisten después de la corrección son de configuración de cabeceras HTTP y de integridad de recursos de terceros (los 5 documentados en la tabla anterior), los cuales no fueron parte del alcance de las correcciones de código de P2 y quedan pendientes como trabajo de endurecimiento (hardening) adicional a nivel de configuración del servidor/middleware.

Nota sobre la comparativa antes/después

Para esta entrega se dispone de un único conjunto de reportes (baseline y full scan), generados ya sobre el código corregido de P2; no se cuenta con un reporte previo a la corrección para una comparativa lado a lado dentro de ZAP. La evidencia de "antes" se documenta de forma indirecta a través del reporte de logs de la Práctica 3 – Parte 2 (mismo entorno, mismos vectores de prueba), que confirma que los intentos de inyección fueron generados y quedaron registrados como fallidos/neutralizados por los controles de código, resultado coherente con la ausencia de hallazgos High en el DAST posterior.

Fase 4 — ZAP en GitHub Actions

Se añadió un job de DAST al pipeline existente (.github/workflows/security.yml), ejecutado después del análisis estático (SonarQube), que levanta la aplicación .NET 10, corre el ZAP Baseline Scan de forma automatizada y publica el reporte como artefacto:

```
dast-zap: name: DAST — OWASP ZAP runs-on: ubuntu-latest needs: sonarqube steps: -
uses: actions/checkout@v4 - name: Setup .NET 10 uses: actions/setup-dotnet@v4 with:
dotnet-version: '10.0.x' - name: Build and run app in background run: | dotnet build
MiAppNetCore.Api/ dotnet run --project MiAppNetCore.Api/ & sleep 10 - name: ZAP Baseline
Scan uses: zapproxy/action-baseline@v0.12.0 with: target: 'http://localhost:5000'
rules_file_name: '.zap/rules.tsv' cmd_options: '-l' - name: Upload ZAP report uses:
actions/upload-artifact@v4 with: name: zap-baseline-report path: report_html.html if: always()
```

Archivo de reglas (.zap/rules.tsv)

Se justifican en el archivo de reglas las alertas conocidas y documentadas en este reporte, para que no bloqueen el pipeline mientras se planifica su corrección:

```
# ID_REGLA ACCION COMENTARIO 10020 WARN Missing Anti-clickjacking Header —
pendiente de corregir 10035 WARN Strict-Transport-Security Header — solo en staging
HTTPS 10038 IGNORE Content Security Policy — evaluar en sprint siguiente
```

Checklist de verificación

Elemento	Estado
Red Docker interna creada y verificada como internal: true	Completado
App vulnerable y ZAP corriendo en la misma red interna	Completado
Baseline scan ejecutado con reporte HTML generado	Completado
Full scan activo ejecutado con reporte HTML y JSON	Completado
Tabla de hallazgos completada (High y Medium)	Completado (0 High, 3 Medium documentados)
Análisis detallado del hallazgo más crítico con request/response	Completado
Correcciones aplicadas y re-escaneo con comparativa	Completado (ver Nota sobre la comparativa)

Elemento	Estado
Job DAST agregado al workflow de GitHub Actions	Completado
Archivo .zap/rules.tsv configurado con justificaciones	Completado
Reporte final: hallazgos antes vs después, metodología, conclusiones	Completado

Nota ética — cierre de la práctica

Al finalizar las pruebas se destruyó el entorno vulnerable, incluyendo contenedores, red interna y volúmenes asociados:

```
docker compose -f pentest-compose.yml down --volumes --remove-orphans
```

Se confirma que los contenedores vulnerables no quedaron corriendo después de concluir la práctica. El alcance de las pruebas se mantuvo exclusivamente dentro de la red interna pentest-net, sin exposición del contenedor vulnerable_app a Internet en ningún momento.

Conclusiones

El análisis DAST con OWASP ZAP contra VulnerableApp, ejecutado tanto en modalidad pasiva (baseline) como activa (full scan) dentro de un entorno Docker aislado, no encontró hallazgos de riesgo High: no se detectaron alertas de SQL Injection, XSS ni otras vulnerabilidades de inyección, lo que es consistente con las correcciones de código aplicadas en la Práctica 2 (consultas parametrizadas, codificación de salida y validación de entrada) y con el registro de eventos analizado en la Práctica 3 — Logs, Parte 2.

Los 5 hallazgos remanentes (3 Medium, 2 Low) corresponden a endurecimiento de configuración —cabeceras de seguridad HTTP ausentes (CSP, X-Frame-Options, X-Content-Type-Options) e integridad de recursos de terceros (SRI)— y no a fallas en la lógica de la aplicación. Se documentaron con su CWE, CVSS estimado y remediación propuesta, y se integró un job de DAST en GitHub Actions con un archivo de reglas que permite gestionar estos hallazgos conocidos sin bloquear el pipeline, dejando planificada su corrección como siguiente iteración de endurecimiento.